EcomApp

Feature Baseline & Frontend QA Report

Evidence-based technical report for the current local feature checkout and Ash's frontend QA contribution.

Verified scope: local feature stage

Project repository: JayIOIO/FinalProject-CI-CD-Automation (reference repository)

Working branch: `feature/frontend-build-label`

Report date: 2026-10-13

Named team roles: Salenga (project lead), Riboroso (backend and CI/CD), Tato (infrastructure), Ash (frontend, QA, documentation)

Roster note: Supplied course guide requires five distinct members; the fifth member's name was not provided. This report does not invent one.

> **Scope and status.** This is not the final deployment report. The checkout has the feature-stage application and local services; it has no team Docker/Compose, deployment proxy configuration, Jenkins pipeline, or integrated smoke-test suite. Final proxy and release evidence must be added by the team.

01 · EXECUTIVE SUMMARY

Purpose, scope, and claims

EcomApp is a marketplace demonstration with buyer and seller experiences backed by separate authentication, product catalog, and order services. The frontend is React with Vite; the APIs are Node.js/Express; data is stored in MySQL. The local feature-stage starter provides the core application modules and a Vite development proxy.

Ash's contribution on this branch supplies a truthful, visible frontend build identifier, build instructions, a manually exercised frontend QA checklist, safe local screenshots, an initial disposable-data integration smoke test, this technical report, and a 15-slide HTML presentation. The UI reads `VITE_BUILD_ID` at bundle-build time. If unset, it displays “unlabeled build” rather than asserting a false deployment version.

Verified in the local environment

- All 15 scenarios in the dated frontend checklist were exercised against the local Vite proxy and local APIs using disposable data.
- Buyer and seller registration/login, catalog search, cart, checkout, order history/cancellation, listing create/edit/delete, seller order view, and fulfillment progression were observed.
- Buyer UI omitted seller navigation; buyer requests to seller-only APIs returned the expected HTTP 403; buyer order history returned HTTP 200.
- Frontend lint, frontend production build, backend JavaScript syntax checks, and the local API/database integration smoke test passed on 2026-10-13.
- All disposable QA users, order data, and products were removed after the run. No passwords or bearer tokens were committed.

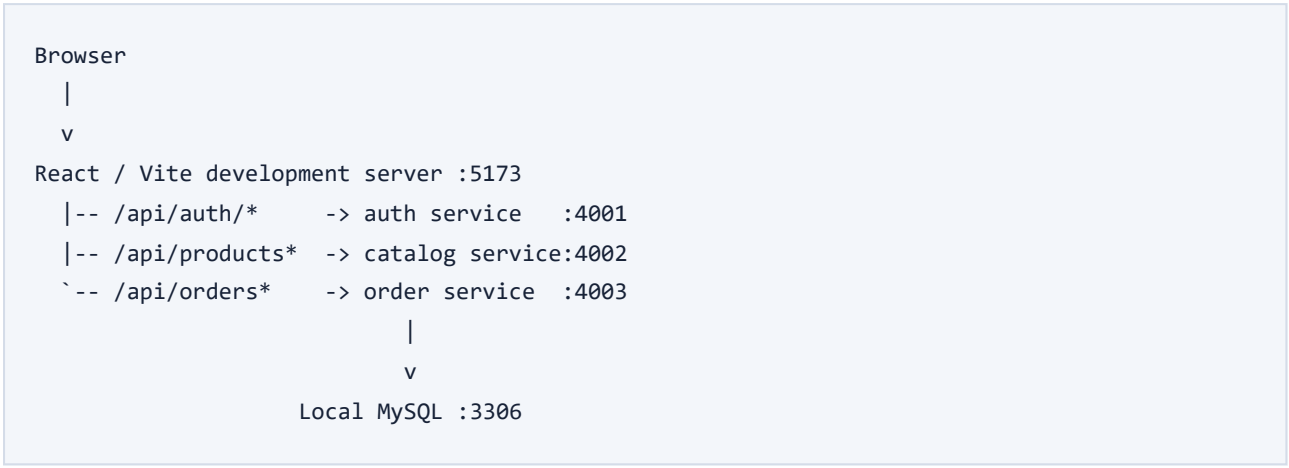
Not verified here

- Operation through the final team reverse proxy. Port 80 responds with a site, but its tested application API routes return 404; HTTPS also presents an untrusted certificate locally.
- Docker/Compose image build/deploy, Jenkins trigger or quality gate, named-volume persistence, rollback, and CI wiring/review of the local smoke test.
- A clean-clone README walkthrough by an unfamiliar teammate, a confirmed five-person roster, or merged/pushed status of the current uncommitted documentation changes.

> This report separates observed behavior from project requirements and planned work. “Not verified” is not a pass.

02 · APPLICATION ARCHITECTURE

Local feature-stage topology



Ports and routes above are from the checked-in local Vite configuration and service configuration. MySQL is an external local prerequisite, not a process started by the root dev script.

Request responsibilities

Component	Observed responsibility	Local interface
React frontend	Authentication forms; buyer search/cart/checkout/history; seller inventory/order views; build ID label	Vite local server :5173
Auth service	Buyer/seller signup and login, signed token issuance, authenticated profile update	`/api/auth/*` via :4001
Catalog service	Public product search and seller-owned inventory mutations	`/api/products*` via :4002
Order service	Buyer checkout/history/cancellation and seller order/fulfillment access	`/api/orders*` via :4003
MySQL	Relational storage for users, products, checkout orders, and order items	Local connection from service configuration

Target topology is a separate phase

The course reference project describes a containerized target with a public NGINX proxy, application containers, persistent MySQL volume, and Jenkins in a separate Compose stack. Those files and runtime evidence are not in this working checkout and are not represented as implemented here.

Buyer, seller, and service boundaries

Buyer workflow

1. Register or log in with buyer role.
2. Search the public catalog; add products to cart.
3. Provide shipping address/payment choice and place an order.
4. Review order details/status and cancel eligible Pending orders.

Seller workflow

1. Register or log in with seller role.
2. Create, edit, view, and delete seller-owned listings.
3. View orders containing owned products.
4. Advance eligible item fulfillment to Processing, Shipped, Delivered.

Role enforcement observed

Frontend navigation and seller screens are conditional on the signed-in role. That presentation guard complements, but does not replace, backend authorization. In the local test a buyer's authenticated requests to `GET /api/products/mine` and `GET /api/orders/seller` returned HTTP 403. Buyer order history returned HTTP 200.

Data integrity paths

The checkout test created an order from selected products and quantities; the buyer could read the order and the owning seller could see the item. Seller fulfillment changed the item status through Processing, Shipped, and Delivered, which then appeared in buyer history. A separate Pending order was cancelled through the UI. These are observed local application behaviors, not a substitute for a formal API contract or cross-service automated test.

> Real test-account identifiers and synthetic shipping details are intentionally omitted from this report. The private session note stores identifiers for QA traceability only.

Stack rationale and persistence boundaries

Technology	Use in current checkout	Reason tied to this application
React	Buyer and seller single-page UI	Composable role-aware screens and local state for cart/forms.
Vite	Frontend development/build tool and local `/api` proxy	Serves the UI locally and routes service prefixes to separate API processes.
Node.js / Express	Three separate HTTP APIs	Keeps authentication, catalog, and order responsibilities distinct.
MySQL	Shared relational persistence	Foreign keys model users, seller products, orders, and order items.
JWT	Authenticated role-bearing requests	Allows protected catalog/order routes to distinguish buyer and seller capabilities.

Data model (checked-in schema)

`users` own products and buyer orders. `products.seller_id` relates inventory to a seller. `checkout_orders.user_id` relates a checkout to its buyer; `order_items` connects each order to products and seller ownership. Foreign keys constrain references; order items cascade with their parent order, while product deletion is restricted when referenced by order history.

Configuration and persistence

Services load local environment configuration from the repository-root `.env`, connect to a local MySQL instance, and use the checked-in initialization SQL for schema/seed setup. The root `npm run dev` starts application processes only; it does not start MySQL. `.env` is private and must not be committed.

A persistent named Docker volume, container restart proof, migration path, and rollback persistence behavior are course requirements for a later integrated phase. None was exercised by this feature-stage checkout.

Reproducing this local stage

1. Install Git, Node.js 22 LTS (or compatible current Node), npm, and local MySQL.
2. Copy `.env.example` to `.env`; set local DB values and a private JWT secret.
3. Initialize the schema using `db/init.sql` against the configured MySQL instance.
4. Run `npm run setup`, then `npm run dev`.
5. Open the frontend and use synthetic, unique buyer/seller accounts.

Checks actually run for this work

Command	Result	Meaning / limitation
`npm run lint`	Passed, 2026-10-13	Frontend source lint; not backend behavior.
`npm run build:frontend`	Passed, 2026-10-13	Vite production bundle built successfully.
`npm run check:services`	Passed, 2026-10-13	Node syntax checks for three service entry points; not service tests.
Manual buyer/seller QA	15/15 scenarios passed locally	See checklist and dated execution record; not final proxy QA.
Cross-service smoke test	`npm run test:smoke` creates unique buyer/seller/product data, checks search, checkout, fulfillment and buyer history, asserts results, and transactionally cleans up. Happy path passed; unavailable order API correctly failed nonzero and cleaned up.	Local pass/fail behavior verified; backend-owner review and CI wiring outstanding

Clean-clone documentation check

The README includes local feature setup and troubleshooting. A fresh-clone walkthrough by a teammate unfamiliar with the system has not been recorded, so documentation usability is not claimed as independently validated.

06 · FRONTEND QA RESULTS

Local browser scenarios

Run on 2026-10-13 in Chromium through the VS Code integrated browser. The UI was at `http://localhost:5173/`; Vite proxied API calls to local services. Service health endpoints returned HTTP 200. Test identities/products were synthetic, disposable, and removed after the run.

Area	Verified observation	Status
Buyer auth	Buyer signup and subsequent login worked; buyer-only navigation appeared.	Pass
Public search	Keyword/category filter `Mechanical` / `Electronics` returned Mechanical Keyboard and excluded Smart Watch.	Pass
Cart	Quantity 2 updated keyboard total to $159.98; remove-item state and post-checkout empty cart were observed.	Pass
Checkout/history	Order was created with synthetic shipping/payment data and rendered in buyer history with amount, item, and status.	Pass
Cancellation	Separate Pending order cancelled; it disappeared from buyer history.	Pass
Seller auth/listings	Seller signup/login and listing create/edit/delete passed; edits appeared in inventory.	Pass
Seller orders	Seller saw an order for its product and updated item status through Processing, Shipped, Delivered.	Pass
Role boundary	Buyer UI omitted seller navigation; buyer API requests to seller-only endpoints returned expected 403.	Pass

> The “15/15” count includes separate UI scenarios for signup/login, cart actions, listing mutations, order view/statuses, and buyer role navigation/API restrictions. Exact actions and results are in `docs/qa-checklist.md`.

Redacted screenshots document the local build identifier in the login and signed-in header. Workflow outcomes are recorded in `docs/evidence/frontend-qa-run-2026-10-13.md`; workflow screenshots were not captured.

07 · BUILD IDENTITY AND EVIDENCE

Version label tied to the bundle source

The sign-in screen and authenticated header display `VITE_BUILD_ID`, compiled by Vite into the frontend bundle. An unset value renders “unlabeled build.” This avoids embedding an unsupported number or release claim into the UI.

Reproducible local build-label check

```
$env:VITE_BUILD_ID = 'local-qa-20261013'
npm run build:frontend
npm --prefix frontend run preview -- --host localhost --port 4173
# Open http://localhost:4173 and inspect the login screen
```

The `local-qa-20261013` label was visually checked in both sign-in and signed-in screens; screenshots are included in the evidence folder. The disposable account used for the authenticated screenshot was deleted after capture.

Release pipeline hand-off

- CI must pass the immutable checked-out commit or release identifier as `VITE_BUILD_ID` before frontend image compilation.
- A runtime environment change cannot modify an already built static bundle.
- Do not use a manually selected build number or use `HEAD` as a claim when uncommitted changes are in the image.
- After merge/deploy, verify the browser label through the final proxy against the actual image/commit used by CI.

Safe evidence inventory

- `docs/evidence/frontend-build-label-login.png` — local QA label at sign-in.
- `docs/evidence/frontend-build-label-signed-in.png` — local QA label in signed-in buyer header with synthetic display name.
- `docs/evidence/frontend-qa-run-2026-10-13.md` — sanitized scenario outcomes and scope caveats.

08 · INTEGRATION STATUS

What the final delivery still needs

Requirement	Current evidence	Status
Final reverse proxy `/` + `/api/...`	Port 80 returned an HTML site; `/api/auth/health`, `/api/products`, and `/api/orders/health` returned 404. Local HTTPS certificate was untrusted.	Not verified / blocker
Dockerfiles and app Compose	No Dockerfiles or Compose file present in this checkout; Docker container inventory did not complete due daemon/tool unavailability.	Not in checkout
Jenkins / automatic SCM trigger	No Jenkinsfile or Jenkins Compose stack present in this checkout.	Not in checkout
Cross-service smoke test	Local CLI flow happy/failure paths passed against local services/database; it is not wired into CI.	Local pass/fail behavior; review/CI pending
Named-volume persistence	No container volume restart/read-back test run.	Not run
Rollback proof	No tested previous image/tag or deployment run available here.	Not run
Final team roster	Four named people appear in supplied role assignments; assignment requires a fifth distinct member.	Confirmation required

Recommended team hand-offs

1. Tato: publish the merged proxy URL/port and the valid local certificate/trust path; retest all `/api/...` prefixes.
2. Riboroso: review the local smoke-test contract/cleanup and wire it into the intended isolated CI test stage.
3. Riboroso/Tato: supply real CI run, failure gate, database volume persistence, and rollback evidence.
4. Salenga: confirm the fifth member and align the final roster/contributions/slides.
5. Ash: repeat UI checklist through the final proxy and replace local-only evidence with sanitized final-proxy evidence.

09 · CONCLUSIONS AND REFERENCES

Evidence-backed conclusion

The current local feature-stage application supports the tested buyer and seller UI journeys. The build label is sourced from a build-time environment value and was confirmed in a local production preview; local lint/build/service syntax checks pass. Disposable QA data was removed, and no test credentials were included in the repository.

This evidence does not establish deployment readiness. The final reverse proxy route set, container orchestration, Jenkins-triggered pipeline, CI integration smoke stage, persistent-volume proof, and rollback run remain outside this checkout or unverified. The final submission narrative should be updated only when owners provide actual merged configuration and dated run results.

Project references

1. Team reference repository: https://github.com/JayIOIO/FinalProject-CI-CD-Automation.
2. Local project setup: `README.md`.
3. Source schema: `db/init.sql`; service routes: `services/auth-service/server.js`, `services/catalog-service/server.js`, `services/order-service/server.js`.
4. Frontend implementation and route proxy: `frontend/src/App.jsx`, `frontend/vite.config.js`.
5. Manual results: `docs/qa-checklist.md` and `docs/evidence/frontend-qa-run-2026-10-13.md`.
6. Course-role and release-gate instructions: supplied Team Execution Guide, sections on Ash's frontend QA, issue workflow, release gates, and technical deliverables.

Submission checks before calling the course project complete

- Confirm the real five-person roster and complete clean-clone instructions.
- Run and archive final-proxy buyer/seller QA, successful/failed pipeline, volume persistence, and rollback evidence.
- Update the slide deck and report with team-supplied actual outcomes; retain all caveats that remain true.

This report contains no passwords, bearer tokens, or private environment values.